

Lecture 14:

Introduction to Deep Learning

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

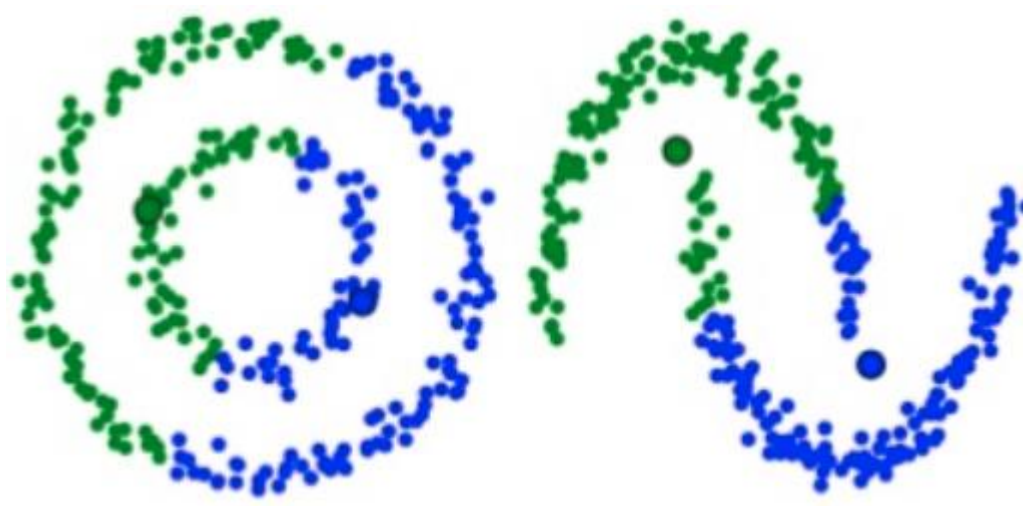
- E-M Clustering Algorithm
- Introduction to Deep learning:
 - Perceptron Algorithm
 - Neural Networks
 - Activation Functions

Clustering Algorithms

- K-Means Clustering
- Mean-Shift Clustering
- Density-Based Spatial Clustering of Applications with Noise (DBSCAN)
- Agglomerative Hierarchical Clustering
- **Expectation-Maximization (EM) Clustering using Gaussian Mixture Models (GMM)**

Expectation-Maximization (EM) Clustering using Gaussian Mixture Models (GMM)

- **The major drawback of K-Means:** use of the mean value for the cluster center.
- *Left-hand side image:* K-Means can't handle this since the mean values are very close together.
- *Right-hand side image:* K-Means also fails when clusters are not circular.

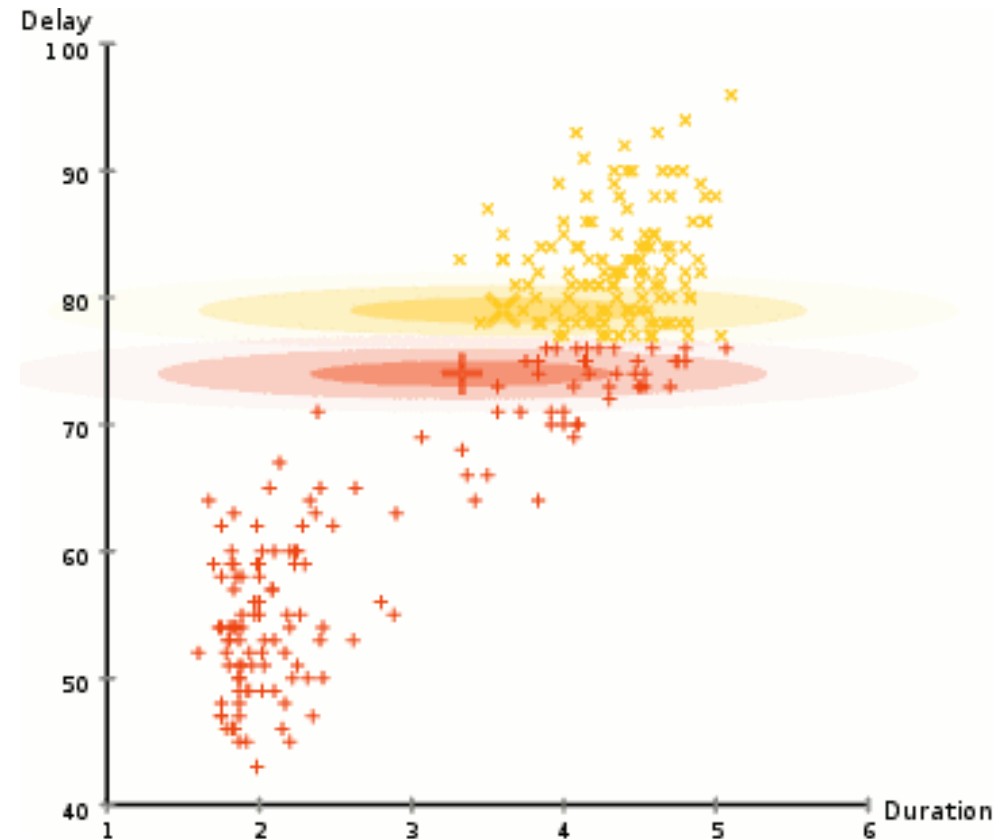


Expectation-Maximization (EM) Clustering using Gaussian Mixture Models (GMM)

- Gaussian Mixture Models is more **flexible** than K-Means.
- GMMs assume that the **data points are Gaussian distributed** (less restrictive assumption than the circular shape in K-Means).
- Two parameters to describe the shape of clusters: the mean and the standard deviation.
- Example of two dimensions: clusters can take any kind of elliptical shape (there is a standard deviation in both the x and y directions). Thus, **each Gaussian distribution is assigned to a single cluster**.

Expectation-Maximization (EM) Clustering using Gaussian Mixture Models (GMM)

- **Objective:** To find the parameters of the Gaussian for each cluster (e.g., the mean and standard deviation).
- **Method:** an optimization algorithm called Expectation-Maximization (EM).

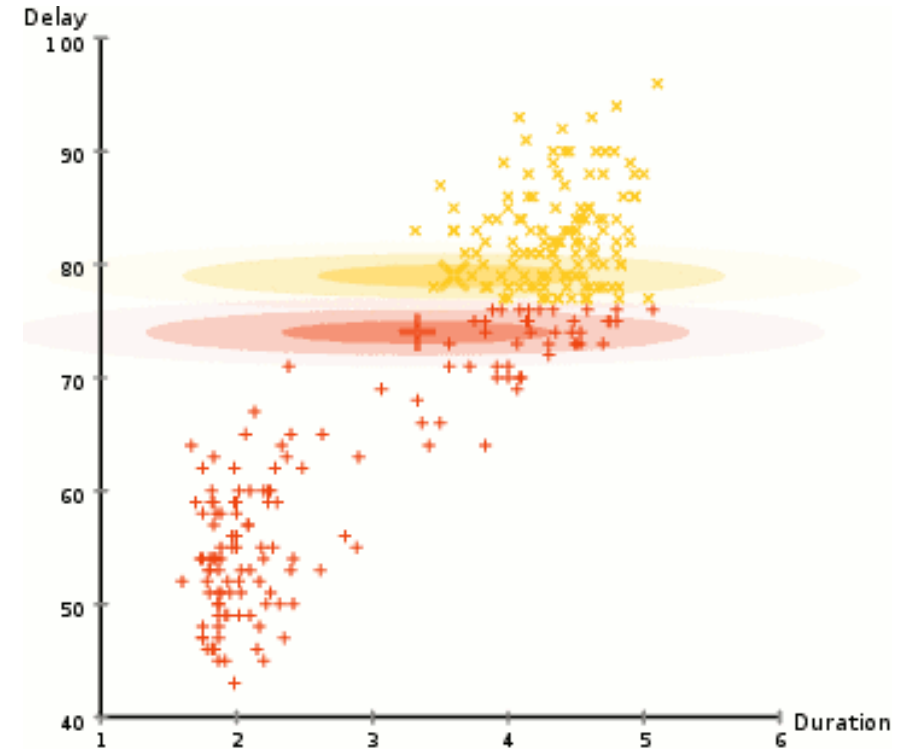


Expectation-Maximization (EM) Clustering Steps

- **Step 1:** choose the number of clusters (similar to K-Means) and randomly initialize the Gaussian distribution parameters for each cluster.
 - Note: use data visualization to identify a good guess
- **Step 2:** Given these distributions for each cluster, compute the probability that each data point fits to a particular cluster. The closer a point is to the Gaussian's center, the more likely it belongs to that cluster.
 - Note: this is compatible with the Gaussian distribution assumption.

Expectation-Maximization (EM) Clustering Steps

- **Step 3:** Based on probabilities, compute a new set of parameters for the Gaussian distributions such that the probabilities of data points within the clusters are maximized.
 - Note: new parameters are computed using a weighted sum of the data point positions. The weights are the probabilities of the data point belonging in that particular cluster.
- Repeat Steps 2 and 3 iteratively until the distributions don't change much from one iteration to another.



Expectation-Maximization (EM) Clustering using Gaussian Mixture Models (GMM)

Advantages:

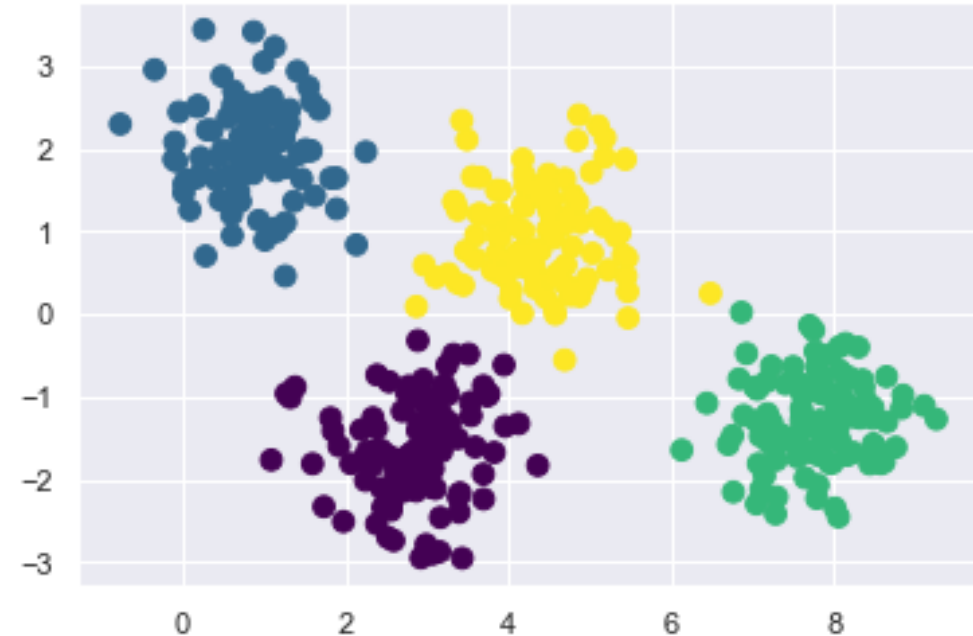
- **More flexible** than K-Means; due to the standard deviation parameter, the clusters can take on any ellipse shape. K-Means is actually a special case of GMM (circular shape where cluster's covariance along all dimensions approaches 0)
- GMMs can have multiple clusters per data point (**mixed membership**). A data point belongs X-percent to class 1 and Y-percent to class 2.

Gaussian Mixture Models

- GMM aims to find a set of multi-dimensional Gaussian probability distributions that best model the input dataset.

```
probs = gmm.predict_proba(X)  
print(probs[:5].round(3))
```

```
[[0.    0.    0.469 0.531]  
 [1.    0.    0.    0.   ]  
 [1.    0.    0.    0.   ]  
 [0.    0.    0.    1.   ]  
 [1.    0.    0.    0.   ]]
```



Expectation-Maximization (EM) Algorithm

- EM algorithm: is a method to **find maximum-likelihood** estimates for model parameters when **your data is incomplete, has missing data points**, or has unobserved (hidden) latent variables.
- It is an iterative way to approximate the maximum likelihood function.

Expectation-Maximization Algorithm

- **Objective:** to use the available observed data to estimate the missing data of the latent variables and then using that data to update the values of the parameters.
- **Two main steps**
 - E-Step (Expectation Step)
 - M-Step (Maximization Step)

The main steps of EM Algorithm

- **Initialization Step:** initialize the parameter values with a set of initial values, then give the set of incomplete observed data to the system with the assumption that the observed data comes from a specific probability distribution.
- **Expectation Step:** Use the observed data to estimate or guess the values of the missing or incomplete data.
- **Maximization Step:** use the complete data generated in the “**Expectation**” step to update the values of the parameters i.e, **update the hypothesis**.

Mathematical Representation of E and M Steps

- Assume that the complete data-set consists of $Z = (X, Y)$ but that **only X is observed**.
- The complete-data log likelihood is then represented by $l(\theta; X, Y)$ where **θ is the unknown parameter** vector for which we wish to find the **MLE**.
- **E-Step**: computes the expected value of $l(\theta; X, Y)$ given the observed data, X , and the current parameter estimate, θ_{old} say.

$$\begin{aligned} Q(\theta; \theta_{old}) &:= E[l(\theta; \mathcal{X}, \mathcal{Y}) \mid \mathcal{X}, \theta_{old}] \\ &= \int l(\theta; \mathcal{X}, y) p(y \mid \mathcal{X}, \theta_{old}) dy \end{aligned}$$

where $p(\cdot \mid X, \theta_{old})$ is the conditional density of Y given the observed data, X , and assuming $\theta = \theta_{old}$

Mathematical Representation of E and M Steps

- **E-Step:** computes the expected value of $l(\theta; X, Y)$ given the observed data, X , and the current parameter estimate, θ_{old} say.

$$\begin{aligned} Q(\theta; \theta_{old}) &:= E[l(\theta; \mathcal{X}, \mathcal{Y}) \mid \mathcal{X}, \theta_{old}] \\ &= \int l(\theta; \mathcal{X}, y) p(y \mid \mathcal{X}, \theta_{old}) dy \end{aligned}$$

where $p(\cdot \mid X, \theta_{old})$ is the conditional density of Y given the observed data, X , and assuming $\theta = \theta_{old}$

- **M-Step:** maximizes over θ the expectation computed in Step E.

$$\theta_{new} := \max_{\theta} Q(\theta; \theta_{old})$$

- Finally, we set $\theta_{old} = \theta_{new}$.

Example of E-M algorithm: A Simple Normal-Mixture Model

- An extremely common application of the EM algorithm: estimate the MLE of normal mixture models (commonly used in clustering).
- Suppose that $X = (X_1, \dots, X_n)$ are IID random variables each with PDF

$$f_x(x) = \sum_{j=1}^m p_j \frac{e^{-(x-\mu_j)^2/2\sigma_j^2}}{\sqrt{2\pi\sigma_j^2}}$$

- where $p_j \geq 0$ for all j and where $\sum p_j = 1$.
- **Objective:** find the MLE estimate for the model parameters including p_j 's, the μ_j 's and the σ_j 's.

Example of E-M algorithm: A Simple Normal-Mixture Model

- This is done with assuming the presence of an additional random variable, Y , where $P(Y = j) = p_j$ for $j = 1, \dots, m$.
- The value of Y determines which of the m normal distributions generates the corresponding value of X . There are n such random variables, $(Y_1, \dots, Y_n) := Y$.

$$f_{x|y}(x_i | y_i = j, \theta) = \frac{1}{\sqrt{2\pi\sigma_j^2}} e^{-(x_i - \mu_j)^2 / 2\sigma_j^2} \quad (1)$$

- where $\theta := (p_1, \dots, p_m, \mu_1, \dots, \mu_m, \sigma_1, \dots, \sigma_m)$: unknown parameter vector.
- The likelihood is given by

$$L(\theta; \mathcal{X}, \mathcal{Y}) = \prod_{i=1}^n p_{y_i} \frac{1}{\sqrt{2\pi\sigma_{y_i}^2}} e^{-(x_i - \mu_{y_i})^2 / 2\sigma_{y_i}^2}.$$

Example of E-M algorithm: A Simple Normal-Mixture Model

- We start the EM algorithm with an initial guess, θ_{old} , and then iterate the E-step and M-step until convergence

- **E-Step:** compute $Q(\theta; \theta_{old}) := E[l(\theta; \mathcal{X}, \mathcal{Y}) \mid \mathcal{X}, \theta_{old}]$

$$Q(\theta; \theta_{old}) = \sum_{i=1}^n \sum_{j=1}^m P(Y_i = j \mid x_i, \theta_{old}) \ln (f_{x|y}(x_i \mid y_i = j, \theta) P(Y_i = j \mid \theta)).$$

- Note that:

$f_{x|y}(x_i \mid y_i = j, \theta)$ is given by (1)

$$P(Y_i = j \mid \theta_{old}) = p_{j,old}.$$

$$P(Y_i = j \mid x_i, \theta_{old}) = \frac{P(Y_i = j, X_i = x_i \mid \theta_{old})}{P(X_i = x_i \mid \theta_{old})} = \frac{f_{x|y}(x_i \mid y_i = j, \theta_{old}) P(Y_i = j \mid \theta_{old})}{\sum_{k=1}^m f_{x|y}(x_i \mid y_i = k, \theta_{old}) P(Y_i = k \mid \theta_{old})}$$

Example of E-M algorithm: A Simple Normal-Mixture Model

- **M-Step:** find new parameters based on maximizing $Q(\theta; \theta_{old})$ by setting partial derivatives, $\partial Q / \partial \theta$, equal to 0 and solving for θ_{new} .

$$\begin{aligned}\mu_{j,new} &= \frac{\sum_{i=1}^n x_i P(Y_i = j \mid x_i, \theta_{old})}{\sum_{i=1}^n P(Y_i = j \mid x_i, \theta_{old})} \\ \sigma_{j,new}^2 &= \frac{\sum_{i=1}^n (x_i - \mu_j)^2 P(Y_i = j \mid x_i, \theta_{old})}{\sum_{i=1}^n P(Y_i = j \mid x_i, \theta_{old})} \\ p_{j,new} &= \frac{1}{n} \sum_{i=1}^n P(Y_i = j \mid x_i, \theta_{old}).\end{aligned}$$

Introduction to Neural Networks

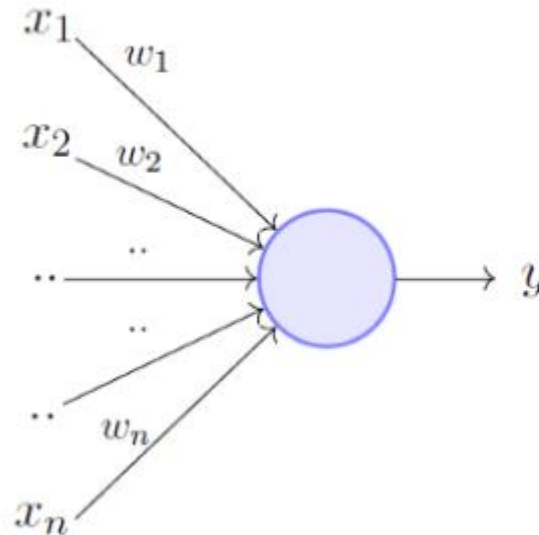
- **Definition:** A neural network is a computational model inspired by the structure of the human brain, consisting of interconnected layers of neurons.
- **Purpose:** Used for complex tasks such as image recognition, natural language processing, and predictive analytics.

Basic Elements of a Neural Network

- Neurons (Nodes)
- Layers (Input, Hidden, Output)
- Weights and Biases
- Activation Functions
- Loss Function
- Optimizer
- Learning Rate

Perceptron Learning Algorithm

- Originally proposed by Frank Rosenblatt in 1943, Later developed and carefully analyzed by Minsky and Papert in 1969.
- The perceptron model takes an input, aggregates it (weighted sum) and returns 1 only if the aggregated sum is more than some threshold else returns 0.



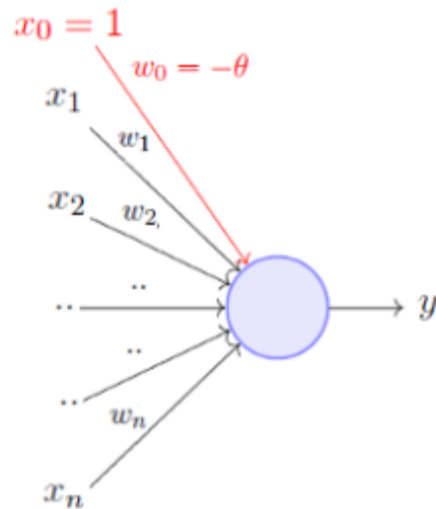
$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i \geq \theta$$
$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i < \theta$$

Rewriting the above,

$$y = 1 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta \geq 0$$
$$= 0 \quad \text{if} \quad \sum_{i=1}^n w_i * x_i - \theta < 0$$

Perceptron Learning Algorithm

- Let's rewrite the threshold and make it a constant input with a variable weight, we will have:



A more accepted convention,

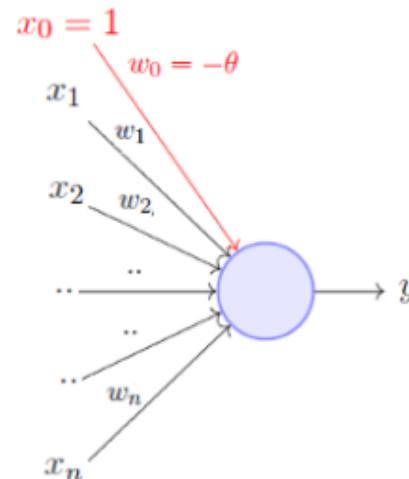
$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$

$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

where, $x_0 = 1$ and $w_0 = -\theta$

Perceptron Learning Algorithm

- A single perceptron can only be used to implement linearly separable functions.
- It takes both real and boolean inputs and associates a set of weights to them, along with a bias (the threshold theta).



A more accepted convention,

$$y = 1 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i \geq 0$$

$$= 0 \quad \text{if} \quad \sum_{i=0}^n w_i * x_i < 0$$

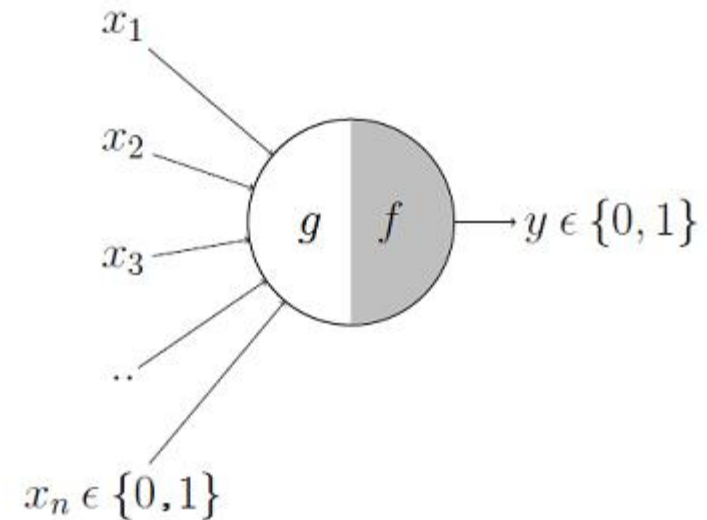
where, $x_0 = 1$ and $w_0 = -\theta$

McCulloch-Pitts Neuron

- The first part, ***g*** takes an input, performs an aggregation and based on the aggregated value the second part, ***f*** makes a decision.

$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$



Perceptron vs McCulloch-Pitts Neuron

McCulloch Pitts Neuron
(assuming no inhibitory inputs)

$$y = 1 \quad \text{if } \sum_{i=0}^n x_i \geq 0$$
$$= 0 \quad \text{if } \sum_{i=0}^n x_i < 0$$

Perceptron

$$y = 1 \quad \text{if } \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if } \sum_{i=0}^n w_i * x_i < 0$$

- What kind of functions can be implemented using a *perceptron*?

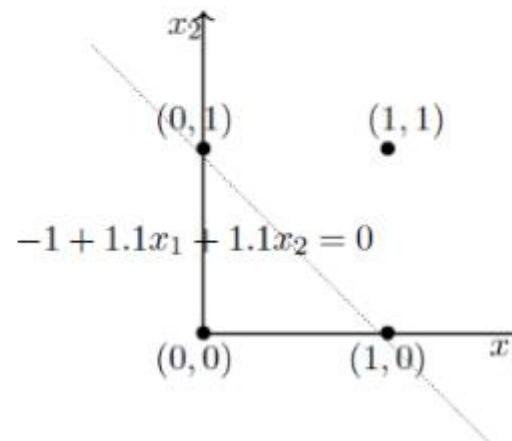
OR Function Using A Perceptron

- Example: Let's use a perceptron to learn an OR function.
- How to learn weights?

x_1	x_2	OR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$

$$\begin{aligned}w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 &\implies w_0 < 0 \\w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 &\implies w_2 > -w_0 \\w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 &\implies w_1 > -w_0 \\w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 &\implies w_1 + w_2 > -w_0\end{aligned}$$

One possible solution is
 $w_0 = -1, w_1 = 1.1, w_2 = 1.1$



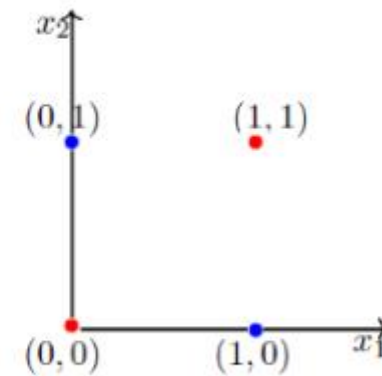
a line that perfectly separates
positive inputs from negative

XOR Function Using A Perceptron

- XOR represents the inequality function, i.e., the output is true if the inputs are not alike otherwise the output is false (must have one or the other but not both)

x_1	x_2	XOR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$

$$\begin{aligned}w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 &\implies w_0 < 0 \\w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 &\implies w_2 > -w_0 \\w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 &\implies w_1 > -w_0 \\w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 &\implies w_1 + w_2 < -w_0\end{aligned}$$



XOR Function Using A Perceptron

- Note: the fourth equation contradicts the second and the third equation.
- Lesson: there are no *perceptron* solutions for non-linearly separated data. So a **single perceptron** cannot learn to separate the data that are non-linear in nature.

$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

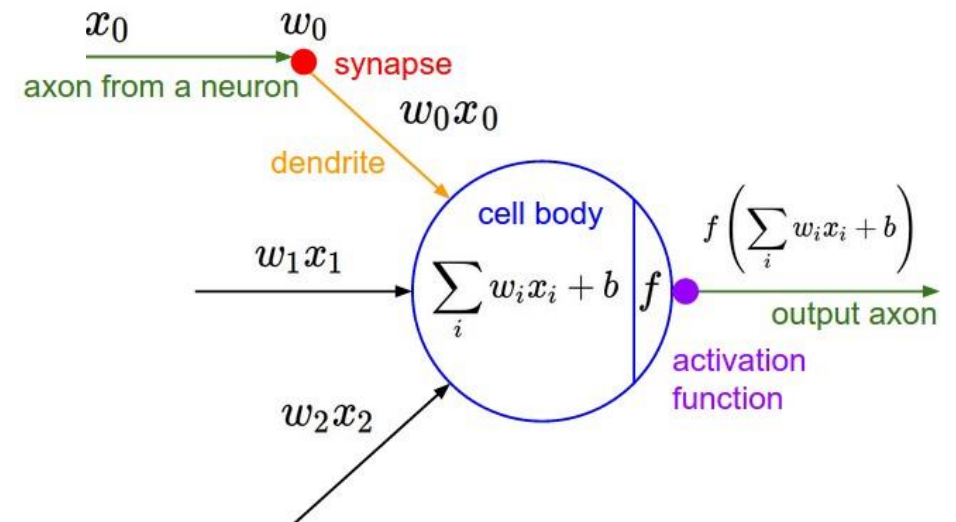
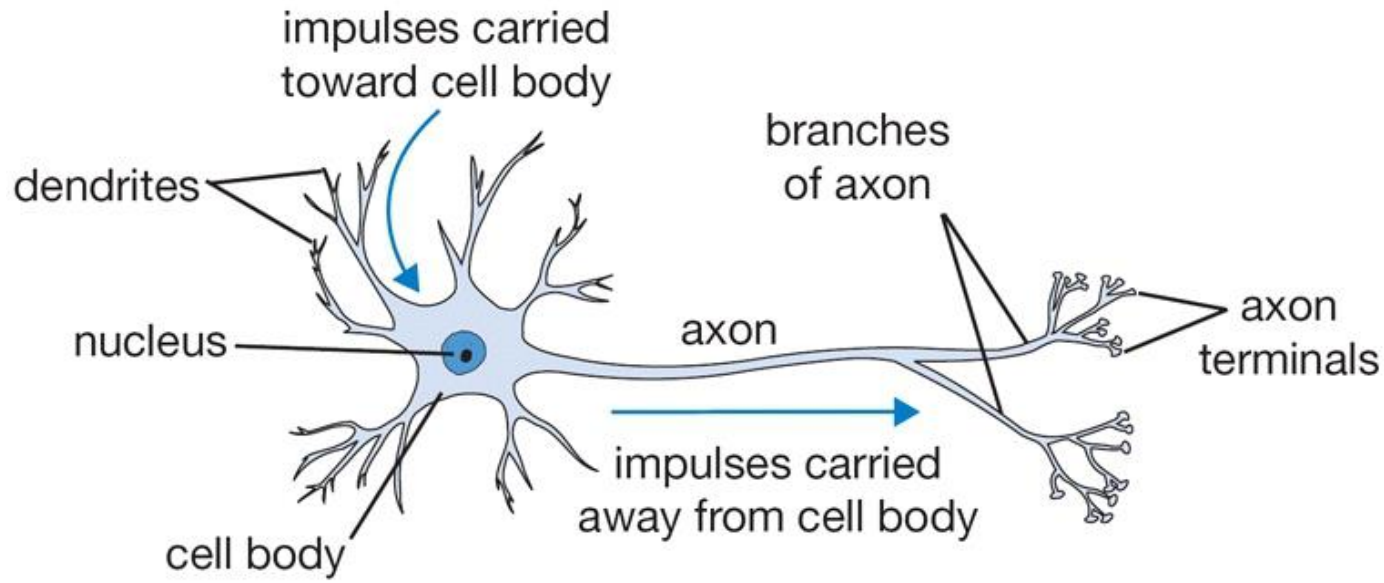
$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 > -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 < -w_0$$

- We need a flexible class of non-linear function with many parameters.
- **Solution:** *Artificial Neural Networks*

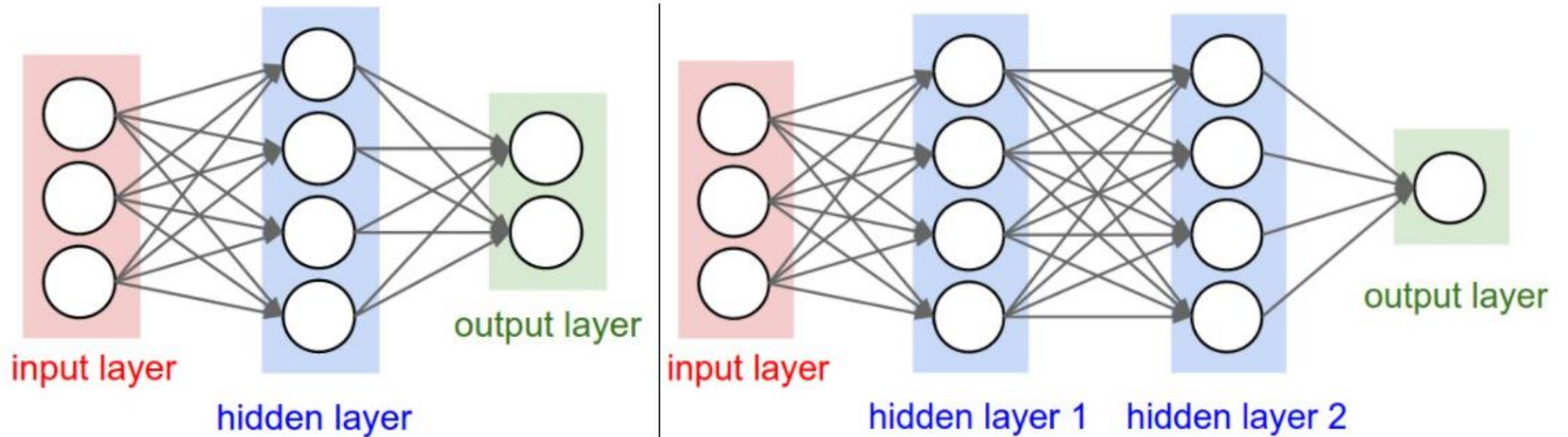
A Biological Neuron vs. Mathematical Model



A Biological Neuron vs. Mathematical Model

- **Neuron:** the basic computational unit of the brain.
- Around 86 billion neurons can be found in the human nervous system.
- Neurons are connected with 10^{14} - 10^{15} **synapses**.
- Each neuron receives input signals from its **dendrites**.
- It produces output signals along its (single) **axon**.
- The axon eventually connects via synapses to dendrites of other neurons.

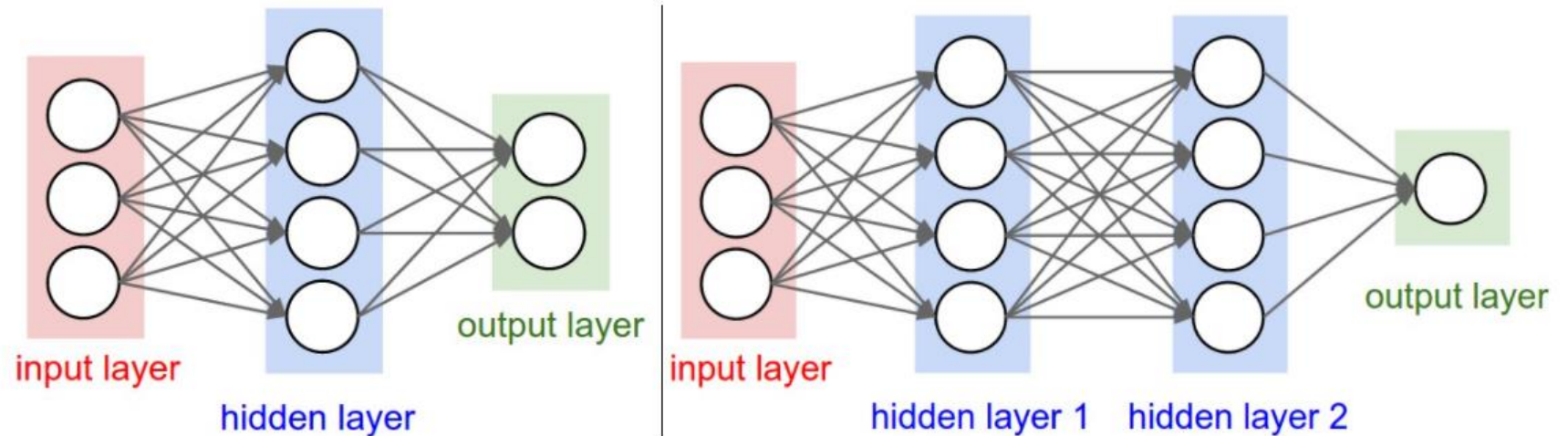
Neural Network Architectures



A 2-layer Neural Network, and a 3-layer network

Neural Network Architectures

- Neural Networks are often organized into separate layers of neurons.
- The most common layer type is the **fully-connected layer**.
- **In fully connected layer** neurons between two adjacent layers are fully pairwise connected. Neurons within a single layer have no connections.



A 2-layer Neural Network, and a 3-layer network

Naming Convention

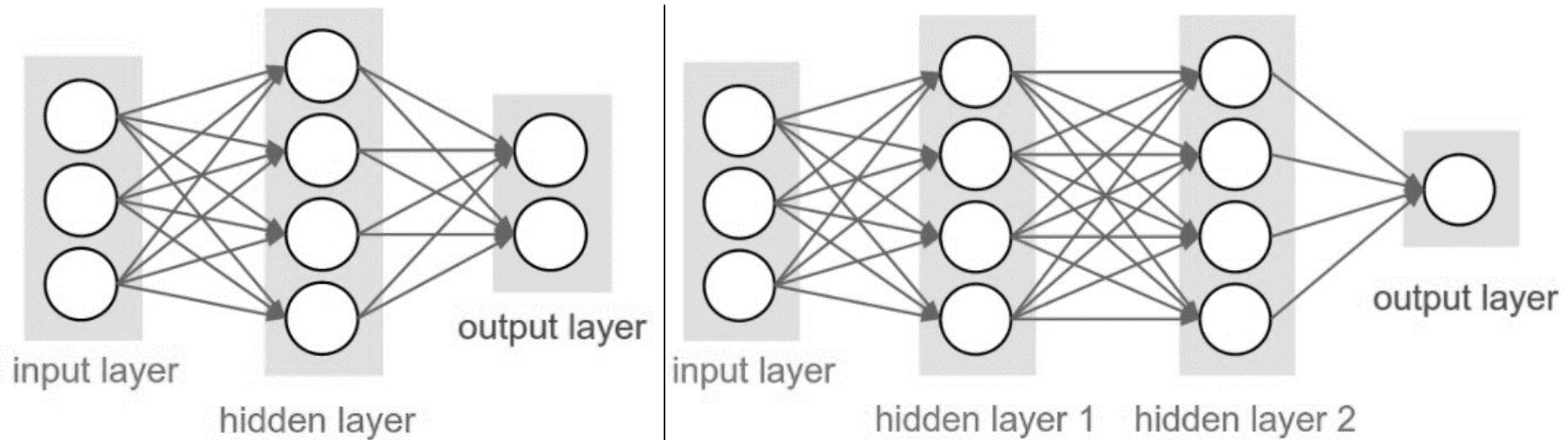
- When we say N-layer neural network, we do not count the input layer.
- A single-layer neural network describes input directly mapped to output (a network with no hidden layers).
- Logistic regression or SVMs are a special case of single-layer Neural Networks.
- *Artificial Neural Networks* (ANN) or *Multi-Layer Perceptrons* (MLP).
- Some people do not like the analogies between Neural Networks and real brains and prefer to refer to neurons as *units*.

Output Layer and Size of the Network

- **Output layer.** do not have an activation function. The last output layer is usually taken to represent the class scores (e.g. in classification), which are real-valued numbers.
- **Sizing neural networks.** The two metrics reflect the size: (1) the number of neurons, or (2) the number of parameters (more common).

Size of the Network

- The left has $4 + 2 = 6$ neurons (not counting the inputs), $[3 \times 4] + [4 \times 2] = 20$ weights and $4 + 2 = 6$ biases (a total of 26 learnable parameters).
- The right network: $4 + 4 + 1 = 9$ neurons, $[3 \times 4] + [4 \times 4] + [4 \times 1] = 12 + 16 + 4 = 32$ weights and $4 + 4 + 1 = 9$ biases (a total of 41 parameters).



Size of Modern Convolutional Networks

- Modern networks include on orders of 100 million parameters and approximately 10-20 layers (hence *deep learning*).

What is an Activation Function?

- A mathematical function applied to each neuron's output to introduce non-linearity into the model.
- This non-linearity helps the network capture and model complex relationships within the data, which would not be possible with purely linear transformations.
- Without activation functions, even a multi-layer neural network would behave like a single-layer network with lack of learning and representing complex data patterns.

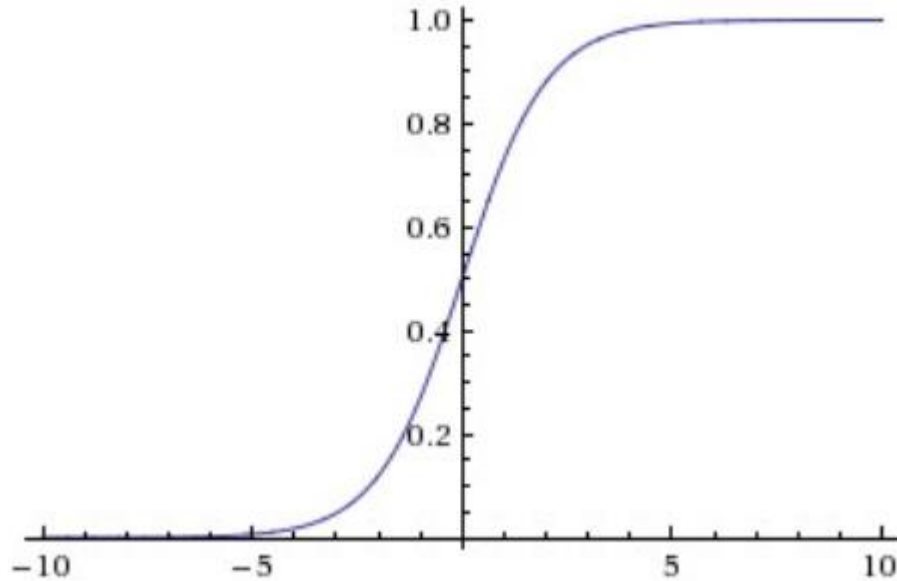
Purpose of Activation Functions?

- **Non-Linearity:** They help networks handle non-linear relationships, essential for tackling complex tasks such as image and speech recognition.
- **Gradient Flow:** They help control the gradient during backpropagation, preventing issues such as gradient vanishing or exploding that can limit learning in deep networks.
- **Bounded Output Range:** Certain activation functions keep neuron outputs within a specified range, which can be beneficial in stabilizing the network's training and generalization.

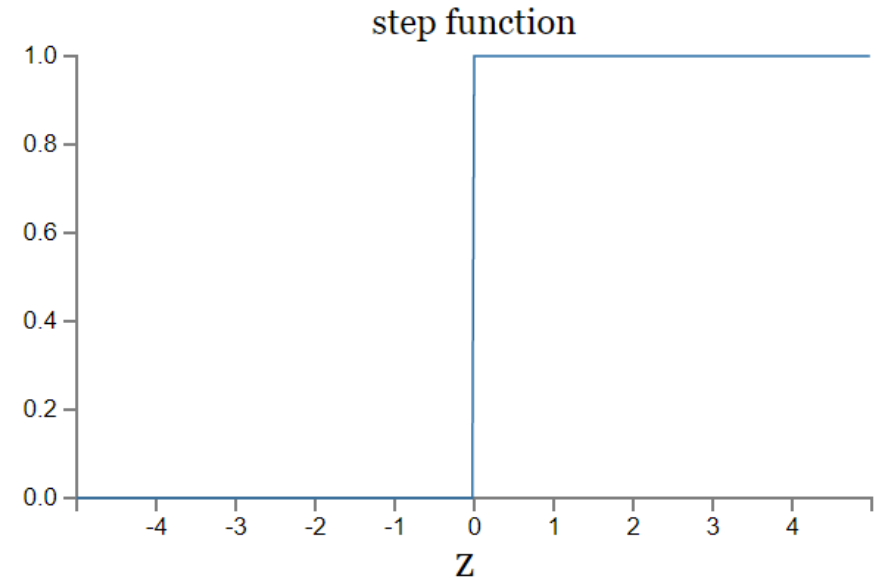
Commonly used activation functions

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- Maxout

Commonly used activation functions: Sigmoid



$$\sigma(x) = 1/(1 + e^{-x})$$



Sigmoid is a smoothed out version of a step function

- **Sigmoid.**
- The sigmoid non-linearity has recently lost their reputation and it is rarely ever used.

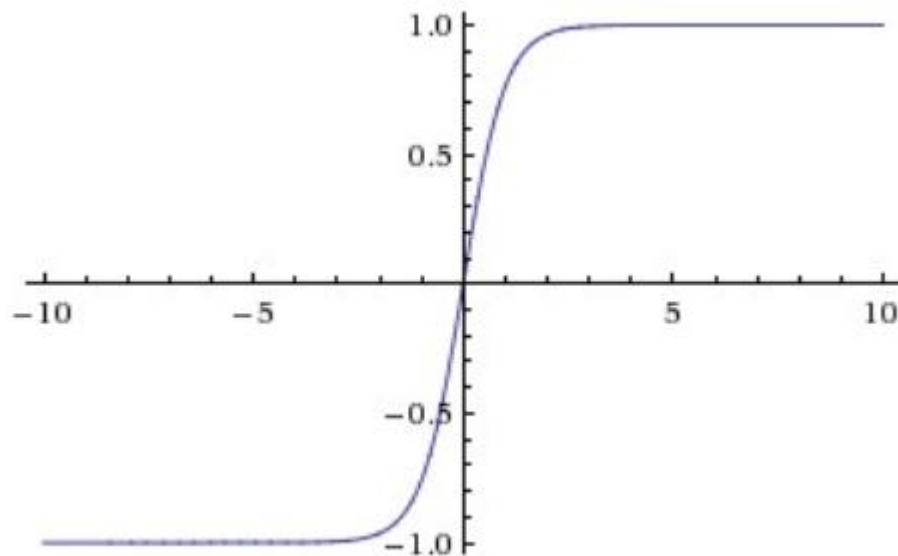
Sigmoid

- Formula:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

- Range (0,1)
- Commonly used in the output layer of binary classification problems.
- **Pros:** Smooth output, interpretable as probability.
- **Cons:** Prone to gradient vanishing, especially in deep networks

Commonly used activation functions: Tanh



$$\tanh(x) = 2\sigma(2x) - 1.$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- **Tanh.** The output is zero-centered. In practice the tanh is preferred to the sigmoid.

Tanh (Hyperbolic Tangent)

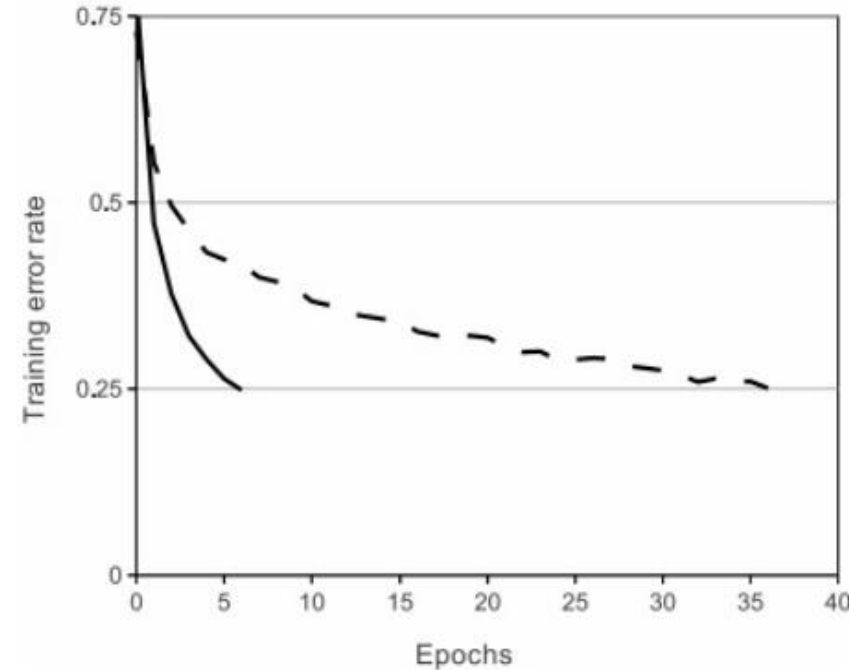
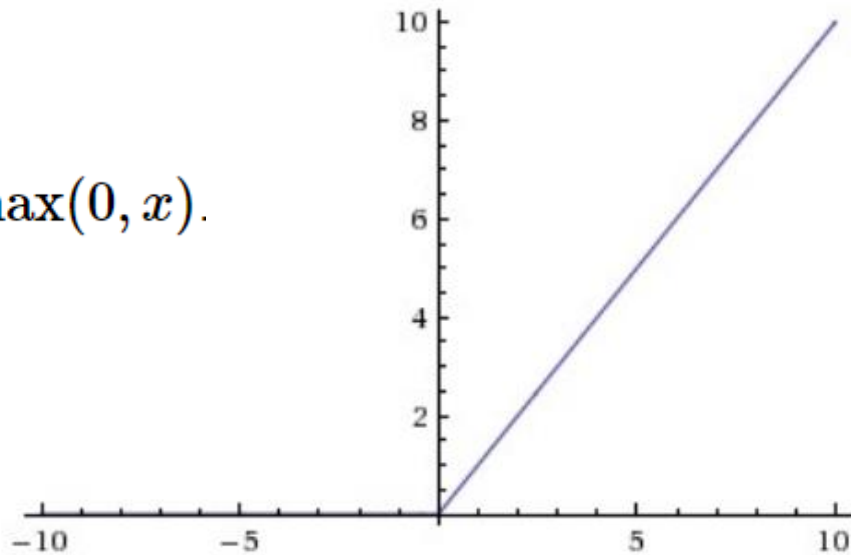
- Formula:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Range (-1,1)
- Often used in hidden layers.
- **Pros:** Zero-centered output, which can lead to faster convergence than Sigmoid.
- **Cons:** Also suffers from the gradient vanishing problem in deep layers.

Commonly used activation functions: ReLU

$$f(x) = \max(0, x).$$



- **Rectified Linear Unit (ReLU).**

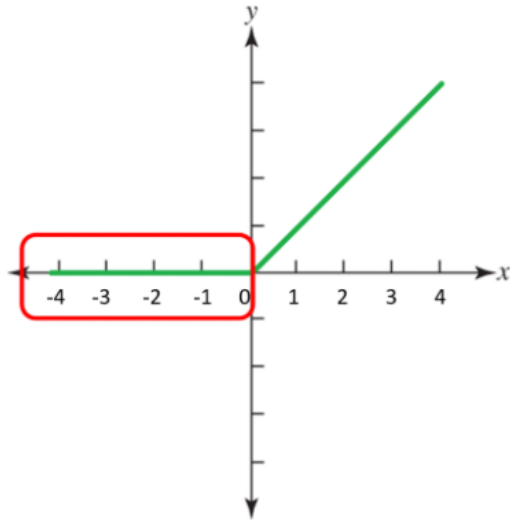
Right figure: The 6x improvement in convergence using ReLU compared to the tanh.

Pros and Cons of ReLU

- It become very popular in the last few years.
- The activation is simply the thresholded at zero.
- It accelerates the convergence of stochastic gradient descent due to its linear, non-saturating form.
- Less computationally intensive compared to tanh/sigmoid neurons that involve expensive operations (exponentials, etc.).
- ReLU units can be fragile during training and “die”.
- For example, a large gradient could cause the weights to update in such a way that the neuron will never activate on any datapoint again.

The Dying ReLU Problem

The dying problem: the scenario when many ReLU neurons only output values of 0 (when the inputs are in the **negative** range)



What causes the Dying ReLU problem?

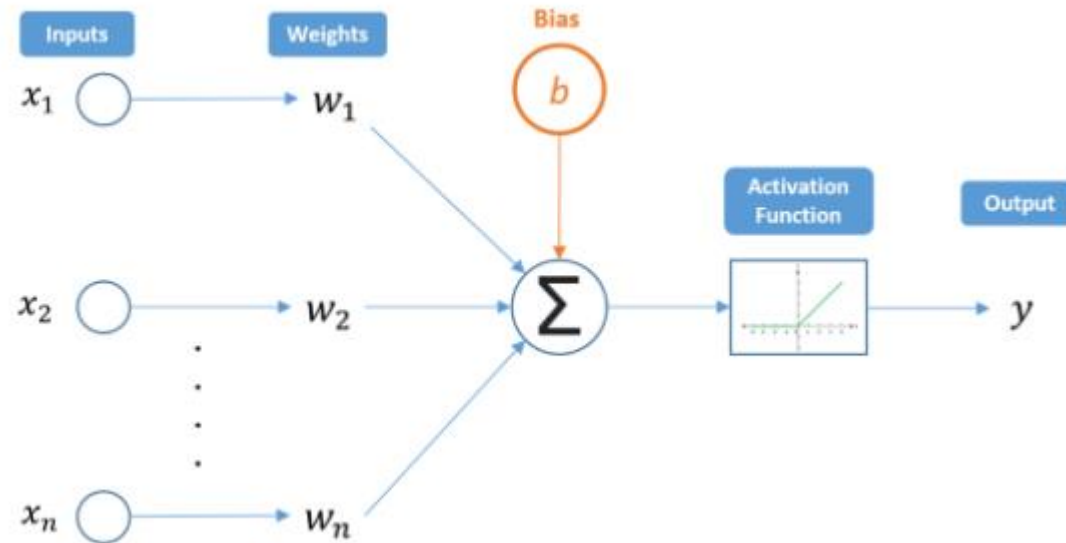
- (1) High learning rate:
- If α is too high, there is a chance that new weights end up in the highly negative value range which result in negative inputs for ReLU.

$$W_{ij}^* = W_{ij} - \alpha \left(\frac{\partial E}{\partial W_{ij}} \right)$$

The diagram shows the weight update equation $W_{ij}^* = W_{ij} - \alpha \left(\frac{\partial E}{\partial W_{ij}} \right)$. Below the equation, four labels are positioned with blue arrows pointing to their respective terms: 'New Weight' points to W_{ij}^* , 'Old Weight' points to W_{ij} , 'Learning Rate' points to α , and 'Derivative of Error with respect to Weight' points to $\left(\frac{\partial E}{\partial W_{ij}} \right)$.

What causes the Dying ReLU problem?

(2) Large negative bias



ReLU (Rectified Linear Unit)

- Formula: $\text{ReLU}(x) = \max(0, x)$
- Range $[0, \infty]$
- **Pros:** Efficient and simple to compute, avoids gradient vanishing, leads to sparse activations (many outputs are zero).
- **Cons:** Can encounter "dying ReLU" problem, where neurons stop learning if they output zero consistently.

Commonly used activation functions: Leaky ReLU

- **Leaky ReLU:**
- An attempt to fix the “dying ReLU” problem of ReLU.
- Instead of zero when $x < 0$, it will have a small negative slope (of 0.01, or so).

$$f(x) = 1(x < 0)(\alpha x) + 1(x \geq 0)(x)$$

- Some have reported success with this activation function, but the results are not always consistent.

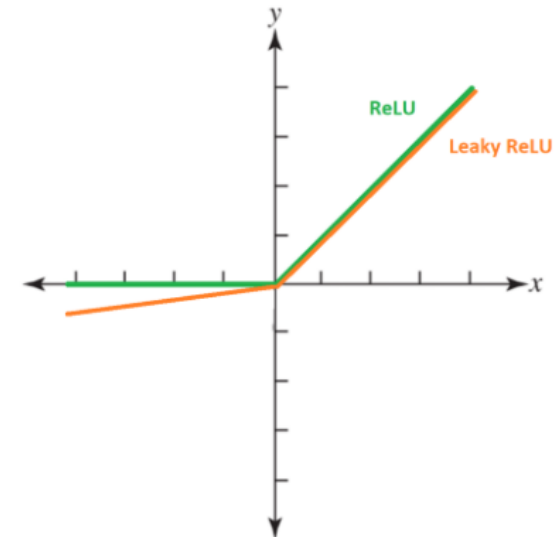
How to solve the Dying ReLU problem?

1. Use of a lower learning rate

2. Variations of ReLU

Leaky ReLU is a common method that generates small negative outputs when input is less than 0.

3. Modification of initialization procedure



Leaky ReLU

- Formula: $\text{Leaky ReLU}(x) = x$ if $x > 0$, otherwise αx
- Range $[-\infty, \infty]$
- **Pros:** Addresses the dying ReLU problem by allowing a small gradient when $x < 0$.
- **Cons:** The value of α is a hyperparameter to tune.

Softmax

- Formula:
$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$
- Range: (0, 1), with all outputs summing to 1
- Used in the output layer for multi-class classification problems.
- **Pros:** Converts logits (raw output scores) to probabilities, which makes it useful for classification.
- **Cons:** Computation can be intensive for large output spaces.

Commonly used activation functions: Maxout

- **Maxout:** Takes the maximum value among the values of “n linear functions”.

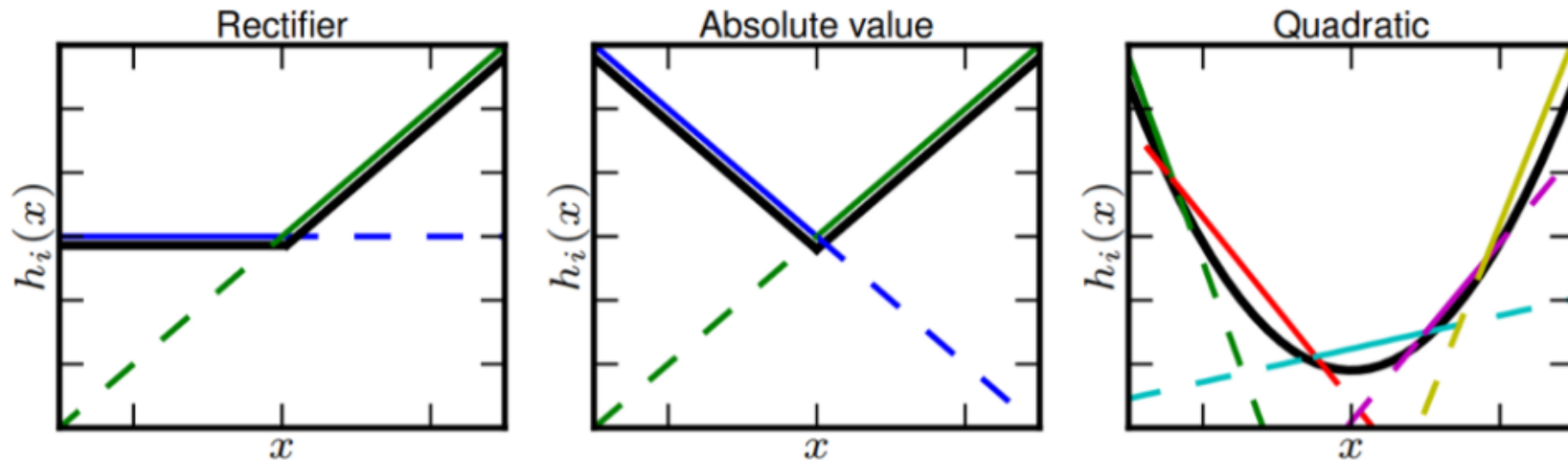
$$h(x) = \max (Z_1, Z_2, \dots, Z_n)$$

$$h(x) = \max (W_1 \cdot x + b_1, W_2 \cdot x + b_2, \dots, W_n \cdot x + b_n)$$

- n is determined beforehand.
- ReLU and Leaky ReLU are a special case of Maxout.
- It has the benefits of a ReLU (linear operation, no saturation) and does not have its drawbacks (dying ReLU).
- Different from the ReLU, it doubles the number of parameters for every single neuron (more parameters).

Commonly used activation functions

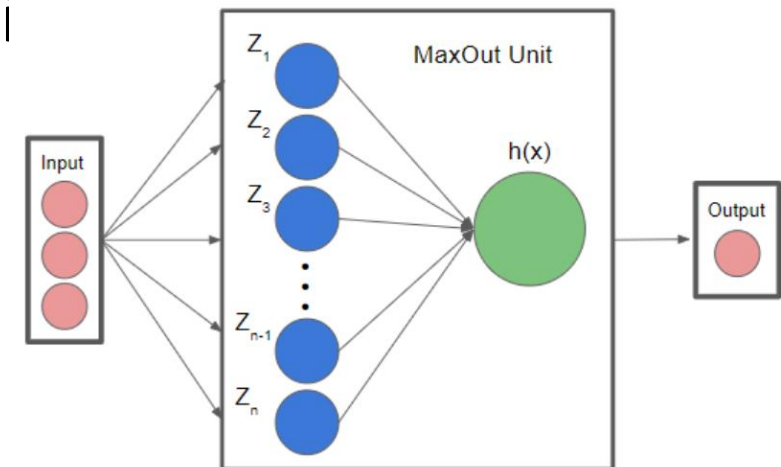
- **Maxout uses Piece-wise linear approximation (PWL):** Approximating a function using multiple linear functions.
- Maxout can approximate arbitrary convex functions (e.g., ReLU, Absolute function, the quadratic function)



$$ReLU = \max(0, x), \quad \text{abs}(x) = \max(x, -x)$$

Commonly used activation functions

- A Maxout neuron has multiple linear functions, each with its own set of weights and biases.
- **One Maxout unit** can estimate a convex piecewise linear function.
- **Proposition:** Any continuous PWL function can be represented as a difference of two convex PWL functions.
- Assume $h_1(x)$ and $h_2(x)$ are two convex functions. Then, the function $g(x)$ is function.



Commonly used activation functions

- Assume $h_1(x)$ and $h_2(x)$ are two convex functions approximated by two Maxout units. Then, $g(x)$ is a **continuous PWL function**.

$$g(x) = h_1(x) - h_2(x)$$

- Conclusion:** a Maxout layer consisting of two Maxout units can approximate any continuous function $f(v)$ arbitrarily well.

Maxout networks are Universal approximators

